

ELITEFORCE.GLOBAL — TEST TECHNIQUE
SENIOR

React Native + Node.js + PostgreSQL + AWS — Projet
MultiServices

Duree : 8 heures minimum
Total : 200 points | Seuil : 140/200
Livraison : 48h apres reception

TEST TECHNIQUE COMPLET

Developpeur Full-Stack Senior — Mobile + Backend

React Native • Node.js • PostgreSQL • Stripe • AWS • Socket.io

AVERTISSEMENT — LISEZ INTEGRALEMENT AVANT DE COMMENCER

Ce test est concu pour evaluer un developpeur SENIOR capable de livrer une application de production.
Il est volontairement complet et exigeant. Un developpeur junior ne peut pas le reussir.
Duree estimee : 8 heures de travail effectif. Prenez le temps necessaire.
Vous avez 48 heures apres reception pour livrer votre repo GitHub.
Chaque module est independant. Commencez par ce que vous maitrisez le mieux.
Le code doit etre propre, commente, et fonctionnel.
Utilisez les technologies imposees. Toute substitution doit etre justifiee dans le README.

RESUME DES 6 MODULES

#	MODULE	COMPETENCES	DUREE	POINTS	NIVEAU
1	Architecture & Setup	GitHub, Structure, Config	45 min	20 pts	Fondation
2	Application Mobile Complète	React Native, Expo, UI/UX	2h30	50 pts	Intermedi aire
3	Backend API REST	Node.js, Express, PostgreSQL	2h00	40 pts	Avance
4	Temps Reel & Notifications	Socket.io, Expo Notifs	1h30	30 pts	Avance
5	Paiement & Sécurité	Stripe, JWT, Validation	1h30	40 pts	Expert
6	Déploiement & Documentation	AWS, CI/CD, README	30 min	20 pts	Expert
TOTAL				200 pts	~8-10h

ELITEFORCE.GLOBAL — TEST TECHNIQUE SENIOR

React Native + Node.js + PostgreSQL + AWS — Projet
MultiServices

Duree : 8 heures minimum
Total : 200 points | Seuil : 140/200
Livraison : 48h apres reception

MODULE 1

Architecture & Setup du Projet

DUREE
45 min

POINTS
20 pts

NIVEAU
Fondation

Objectif

Avant d'ecrire une seule ligne de code metier, vous devez prouver que vous savez organiser un projet professionnel. Un mauvais setup = un projet qui explose en production.

1.1 — Structure GitHub (8 pts)

Creez un repo GitHub public nomme exactement :

```
eliteforce-test-[votre-prenom]-[votre-nom]
```

Organisez-le avec cette structure EXACTE :

```
eliteforce-test-prenom-nom/  
├── mobile/                               ← App React Native Expo  
│   ├── src/  
│   │   ├── screens/  
│   │   ├── components/  
│   │   ├── navigation/  
│   │   ├── services/                   ← Appels API  
│   │   ├── store/                     ← Redux  
│   │   └── utils/  
│   ├── app.json  
│   └── package.json  
├── backend/                             ← API Node.js  
│   ├── src/  
│   │   ├── routes/  
│   │   ├── controllers/  
│   │   ├── models/  
│   │   ├── middleware/  
│   │   └── config/  
│   ├── prisma/  
│   │   └── schema.prisma  
│   └── package.json  
├── README.md                           ← OBLIGATOIRE  
└── .gitignore                           ← node_modules, .env
```

1.2 — Configuration Environnement (6 pts)

Creez les fichiers .env.example pour mobile ET backend avec toutes les variables necessaires :

```
// backend/.env.example  
DATABASE_URL=postgresql://user:password@localhost:5432/eliteforce  
JWT_SECRET=votre_secret_jwt  
JWT_EXPIRES_IN=7d  
STRIPE_SECRET_KEY=sk_test_...
```

ELITEFORCE.GLOBAL — TEST TECHNIQUE SENIOR

React Native + Node.js + PostgreSQL + AWS — Projet
MultiServices

Duree : 8 heures minimum
Total : 200 points | Seuil : 140/200
Livraison : 48h apres reception

```
PORT=3000
```

```
// mobile/.env.example
API_URL=http://localhost:3000
STRIPE_PUBLISHABLE_KEY=pk_test_...
GOOGLE_MAPS_API_KEY=votre_cle_google
```

1.3 — README.md (6 pts)

Le README doit contenir OBLIGATOIREMENT :

- Description du projet en 3-4 lignes
- Prerequis (Node version, Expo CLI, PostgreSQL)
- Instructions d'installation pas a pas
- Commandes pour lancer mobile ET backend
- Variables d'environnement expliquees
- Screenshots de l'app (minimum 3)

CRITERE D'EVALUATION	DESCRIPTION	POINTS
Structure dossiers respectee	Architecture mobile + backend correcte	5 pts
Fichiers .env.example complets	Toutes variables presentes et documentees	4 pts
README clair et complet	Installation possible sans aide	5 pts
.gitignore correct	node_modules et .env non commites	3 pts
Commits propres et descriptifs	Messages de commit clairs en francais	3 pts

ELITEFORCE.GLOBAL — TEST TECHNIQUE SENIOR

React Native + Node.js + PostgreSQL + AWS — Projet MultiServices

Duree : 8 heures minimum
Total : 200 points | Seuil : 140/200
Livraison : 48h apres reception

MODULE 2

Application Mobile Complète

DUREE
2h 30min

POINTS
50 pts

NIVEAU
Intermediaire

Contexte

Vous developpez le MVP de l'application client de MultiServices. L'utilisateur doit pouvoir s'inscrire, se connecter, chercher des services, et voir son profil.

2.1 — Navigation & Architecture (10 pts)

Implementez la navigation avec React Navigation v6 :

```
npm install @react-navigation/native @react-navigation/stack
npm install @react-navigation/bottom-tabs
```

Structure de navigation OBLIGATOIRE :

```
AppNavigator
├── AuthStack (si non connecte)
│   ├── LoginScreen
│   ├── RegisterScreen
│   └── ForgotPasswordScreen
└── MainStack (si connecte)
    └── BottomTabs
        ├── HomeScreen (icone maison)
        ├── SearchScreen (icone loupe)
        ├── BookingsScreen (icone calendrier)
        └── ProfileScreen (icone personne)
```

2.2 — Ecran d'Inscription (10 pts)

Creez un formulaire d'inscription complet avec les champs suivants :

- Prenom + Nom
- Email (validation format email)
- Telephone (validation format +212XXXXXXXXXX pour le Maroc)
- Mot de passe (minimum 8 caracteres, 1 majuscule, 1 chiffre)
- Confirmation mot de passe (doit etre identique)
- Case a cocher CGU (obligatoire)

Comportement attendu :

1. Validation en temps reel — message d'erreur sous chaque champ
2. Bouton 'S'inscrire' desactive si formulaire invalide
3. Appel API POST /api/auth/register si formulaire valide
4. Spinner de chargement pendant l'appel API
5. Navigation vers HomeScreen si succes

ELITEFORCE.GLOBAL — TEST TECHNIQUE SENIOR

React Native + Node.js + PostgreSQL + AWS — Projet
MultiServices

Duree : 8 heures minimum
Total : 200 points | Seuil : 140/200
Livraison : 48h apres reception

6. Message d'erreur si email deja utilise

2.3 — Ecran de Connexion (5 pts)

Ecran de connexion avec :

- Champs email et mot de passe
- Afficher/masquer le mot de passe
- Connexion via API POST /api/auth/login
- Sauvegarde du JWT token avec AsyncStorage
- Gestion des erreurs (mauvais identifiants)

2.4 — Ecran d'Accueil (10 pts)

L'ecran principal de l'application doit afficher :

- Message de bienvenue avec le prenom de l'utilisateur connecte
- Barre de recherche rapide (non fonctionnelle, juste UI)
- Section 'Categories de services' avec icones (grille 2x3 minimum) :
 - Menage | Plomberie | Electricite
 - Jardinage | Demenagement | Peinture
- Section 'Services populaires' — FlatList horizontale
 - Chaque item : image placeholder + nom service + prix de base
- Scroll vertical de tout l'ecran
- Design soigne et professionnel

2.5 — Ecran de Recherche avec Filtres (10 pts)

Ecran de recherche avance :

- Barre de recherche avec debounce de 400ms
 - Ne pas appeler l'API a chaque frappe — attendre 400ms d'inactivite
- Filtres par categorie (chips horizontaux selectionables)
- Filtre de prix : slider min/max
- Filtre de note : etoiles minimum
- Appel API GET /api/services?search=X&category=Y&minPrice=Z
- Affichage resultats en FlatList
- Message 'Aucun resultat' si liste vide
- Pull-to-refresh pour recharger

2.6 — Gestion d'Etat avec Redux Toolkit (5 pts)

Implementez Redux Toolkit avec au minimum ces slices :

```
// store/authSlice.js — Exemple attendu
const authSlice = createSlice({
  name: 'auth',
  initialState: { user: null, token: null, loading: false },
```

ELITEFORCE.GLOBAL — TEST TECHNIQUE SENIOR

React Native + Node.js + PostgreSQL + AWS — Projet
MultiServices

Duree : 8 heures minimum

Total : 200 points | Seuil : 140/200

Livraison : 48h apres reception

```
reducers: {  
  setUser: (state, action) => { state.user = action.payload },  
  setToken: (state, action) => { state.token = action.payload },  
  logout: (state) => { state.user = null; state.token = null }  
}  
})
```

- authSlice : user, token, isAuthenticated
- servicesSlice : liste, loading, error, filtres actifs

CRITERE D'EVALUATION	DESCRIPTION	POINTS
Navigation correcte	AuthStack + MainStack + BottomTabs fonctionnels	10 pts
Formulaire inscription	Validation complete + appel API + gestion erreurs	10 pts
Ecran connexion	JWT sauvegarde + navigation post-connexion	5 pts
Ecran accueil	Categories + FlatList horizontale + design	10 pts
Ecran recherche	Debounce + filtres + resultats + pull-to-refresh	10 pts
Redux Toolkit	Slices corrects + store configure + useSelector/useDispatch	5 pts

ELITEFORCE.GLOBAL — TEST TECHNIQUE SENIOR

React Native + Node.js + PostgreSQL + AWS — Projet MultiServices

Duree : 8 heures minimum
Total : 200 points | Seuil : 140/200
Livraison : 48h apres reception

MODULE 3

Backend API REST Complet

DUREE
2h 00min

POINTS
40 pts

NIVEAU
Avance

Contexte

Le backend est le coeur de l'application. Il doit etre solide, securise, et bien structure. Vous utilisez Node.js + Express + Prisma + PostgreSQL.

3.1 — Schema Prisma (10 pts)

Creez le schema Prisma complet dans backend/prisma/schema.prisma :

```
// Les 4 modeles OBLIGATOIRES
model User {
  id          Int           @id @default(autoincrement())
  email       String        @unique
  password    String
  firstName   String
  lastName    String
  phone       String?
  role        Role          @default(CLIENT)
  createdAt   DateTime      @default(now())
  bookings    Booking[]
}
```

```
model Service {
  id          Int           @id @default(autoincrement())
  name        String
  description  String
  category     String
  basePrice   Float
  duration     Int           ← en minutes
  rating       Float        @default(0)
  available    Boolean      @default(true)
  bookings    Booking[]
}
```

```
model Booking {
  id          Int           @id @default(autoincrement())
  userId      Int
  serviceId   Int
  status      BookingStatus @default(PENDING)
  scheduledAt DateTime
  address     String
  totalPrice  Float
  createdAt   DateTime      @default(now())
  user        User          @relation(fields: [userId], references: [id])
  service     Service       @relation(fields: [serviceId], references: [id])
}
```

```
enum Role { CLIENT PROVIDER ADMIN }
```

ELITEFORCE.GLOBAL — TEST TECHNIQUE SENIOR

React Native + Node.js + PostgreSQL + AWS — Projet MultiServices

Duree : 8 heures minimum
Total : 200 points | Seuil : 140/200
Livraison : 48h apres reception

```
enum BookingStatus { PENDING CONFIRMED IN_PROGRESS COMPLETED CANCELLED }
```

3.2 — Endpoints Authentication (8 pts)

Implementez les routes d'authentification :

METHODE	ENDPOINT	BODY	REPOSE
POST	/api/auth/register	firstName, lastName, email, password, phone	{ user, token } + 201
POST	/api/auth/login	email, password	{ user, token } + 200
GET	/api/auth/me	Header: Authorization: Bearer token	{ user } + 200 ou 401
POST	/api/auth/logout	Header: Authorization	{ message } + 200

3.3 — Endpoints Services (10 pts)

METHODE	ENDPOINT	DESCRIPTION	AUTH REQUISE
GET	/api/services	Liste + filtres search, category, minPrice, maxPrice, rating	Non
GET	/api/services/:id	Detail d'un service + infos associees	Non
POST	/api/services	Creer un service — ADMIN seulement	Oui — ADMIN
PUT	/api/services/:id	Modifier un service — ADMIN seulement	Oui — ADMIN
DELETE	/api/services/:id	Supprimer — ADMIN seulement	Oui — ADMIN

3.4 — Endpoints Reservations (8 pts)

- POST /api/bookings — Creer une reservation (CLIENT authentifie)
 - Verifier que le service existe et est disponible
 - Calculer le prix total automatiquement
 - Retourner la reservation creee avec les infos du service
- GET /api/bookings/me — Mes reservations (CLIENT authentifie)
 - Inclure les infos du service dans chaque reservation
 - Trier par date de creation descendante
- PUT /api/bookings/:id/cancel — Annuler une reservation
 - Seulement si status = PENDING et appartient au user
 - Passer status a CANCELLED
- GET /api/bookings — Toutes les reservations (ADMIN seulement)

ELITEFORCE.GLOBAL — TEST TECHNIQUE SENIOR

React Native + Node.js + PostgreSQL + AWS — Projet
MultiServices

Duree : 8 heures minimum
Total : 200 points | Seuil : 140/200
Livraison : 48h apres reception

3.5 — Middleware de Securite (4 pts)

Implementez ces middlewares OBLIGATOIRES :

- `authenticate` — Verifie le JWT token dans le header Authorization
- `authorize(roles)` — Verifie que l'utilisateur a le bon role
- `validateRequest` — Validation des donnees avec `express-validator` ou `Joi`
- `errorHandler` — Middleware global de gestion des erreurs

```
// Exemple d'utilisation attendue
router.post('/services', authenticate, authorize(['ADMIN']), validateRequest,
createService)
```

CRITERE D'EVALUATION	DESCRIPTION	POINTS
Schema Prisma complet	4 modeles + enums + relations corrects	10 pts
Auth Register + Login	Hash password bcrypt + JWT genere correctement	5 pts
Middleware authenticate	Verification JWT + user attache a req	3 pts
GET /services avec filtres	Filtres Prisma fonctionnels (where, search)	5 pts
CRUD services complet	POST/PUT/DELETE avec auth ADMIN	5 pts
POST /bookings	Verification service + calcul prix + creation	5 pts
GET /bookings/me	Reservations user avec infos service incluses	4 pts
Gestion erreurs globale	Middleware errorHandler + codes HTTP corrects	3 pts

ELITEFORCE.GLOBAL — TEST TECHNIQUE SENIOR

React Native + Node.js + PostgreSQL + AWS — Projet MultiServices

Duree : 8 heures minimum
Total : 200 points | Seuil : 140/200
Livraison : 48h apres reception

MODULE 4

Temps Reel & Notifications Push

DUREE
1h 30min

POINTS
30 pts

NIVEAU
Avance

Contexte

MultiServices est une application de mise en relation en temps reel. Le client doit recevoir des notifications quand son prestataire est en route, arrive, ou termine la mission.

4.1 — Socket.io Backend (12 pts)

Ajoutez Socket.io a votre serveur Express :

```
npm install socket.io
```

Implementez ces evenements Socket.io cote serveur :

EVENEMENT	DIRECTION	DESCRIPTION
connection	Client → Serveur	Connexion d'un utilisateur — logger l'ID socket
join-booking-room	Client → Serveur	Rejoindre la room d'une reservation specifique
booking-status-update	Serveur → Client	Statut de la reservation change (CONFIRMED, IN_PROGRESS...)
provider-location	Serveur → Client	Position GPS du prestataire mise a jour
mission-completed	Serveur → Client	Mission terminee — declencher ecran de notation
disconnect	Client → Serveur	Nettoyage a la deconnexion

4.2 — Socket.io Mobile (8 pts)

Connectez l'application mobile au serveur Socket.io :

```
npm install socket.io-client
```

- Connexion au serveur avec le token JWT dans les headers
- Rejoindre automatiquement la room de la reservation active
- Ecouter booking-status-update et mettre a jour l'interface
- Afficher la position du prestataire sur une carte en temps reel
- Afficher une notification in-app quand la mission est terminee

```
// Exemple de connexion attendu
const socket = io(API_URL, {
  auth: { token: userToken },
  transports: ['websocket']
})
socket.on('booking-status-update', (data) => {
```

ELITEFORCE.GLOBAL — TEST TECHNIQUE SENIOR

React Native + Node.js + PostgreSQL + AWS — Projet MultiServices

Duree : 8 heures minimum

Total : 200 points | Seuil : 140/200

Livraison : 48h apres reception

```
dispatch(updateBookingStatus(data))
})
```

4.3 — Notifications Push Expo (10 pts)

Implementez les notifications push avec expo-notifications :

```
expo install expo-notifications expo-device
```

- Demander la permission de notifications au demarrage
- Recuperer le token Expo Push et l'envoyer au backend
- Backend : sauvegarder le pushToken dans le modele User
- Backend : envoyer une notification via API Expo quand :
 - Reservation confirmee → 'Votre reservation est confirmee !'
 - Prestataire en route → 'Votre prestataire arrive dans 10 min'
 - Mission terminee → 'Mission terminee — Notez votre prestataire'
- Gestion des notifications recues en foreground ET background

```
// Envoi notification via Expo Push API
const message = {
  to: user.pushToken,
  sound: 'default',
  title: 'Reservation confirmee',
  body: 'Votre prestataire sera la le 15/03 a 10h00',
  data: { bookingId: booking.id }
}
```

CRITERE D'EVALUATION	DESCRIPTION	POINTS
Serveur Socket.io configure	Integration avec Express + CORS socket	4 pts
Evenements serveur implementes	Tous les evenements definis et fonctionnels	8 pts
Client mobile connecte	Connexion avec JWT + ecoute des evenements	8 pts
Permission notifications	Demande + recuperation pushToken	3 pts
Envoi notifications backend	3 scenarios de notification implementes	5 pts
Reception notifications mobile	Foreground + background geres	2 pts

ELITEFORCE.GLOBAL — TEST TECHNIQUE SENIOR

React Native + Node.js + PostgreSQL + AWS — Projet MultiServices

Duree : 8 heures minimum
Total : 200 points | Seuil : 140/200
Livraison : 48h apres reception

MODULE 5

Paie ment Stripe & Sécurité Avancée

DUREE
1h 30min

POINTS
40 pts

NIVEAU
Expert

Contexte

Le paiement est la fonction la plus critique de MultiServices. Une erreur ici = perte d'argent reel. Vous devez implementer Stripe de maniere securisee et professionnelle.

5.1 — Architecture Paiement Stripe (15 pts)

Implementez le flux de paiement COMPLET selon l'architecture Stripe recommandee :

ETAPE 1 — Backend cree un PaymentIntent :

```
POST /api/payments/create-intent
Body: { bookingId: number }
Response: { clientSecret: string, amount: number }
```

ETAPE 2 — Mobile confirme le paiement :

```
// Utilisation de @stripe/stripe-react-native
const { error, paymentIntent } = await confirmPayment(clientSecret, {
  paymentMethodType: 'Card',
  paymentMethodData: { billingDetails: { email: user.email } }
})
```

ETAPE 3 — Backend confirme via Webhook :

```
POST /api/payments/webhook
// Verifier la signature Stripe OBLIGATOIREMENT
const event = stripe.webhooks.constructEvent(
  req.body, sig, process.env.STRIPE_WEBHOOK_SECRET
)
```

ETAPE 4 — Backend met a jour la reservation :

```
// Sur event 'payment_intent.succeeded'
await prisma.booking.update({
  where: { stripePaymentId: paymentIntent.id },
  data: { status: 'CONFIRMED', paidAt: new Date() }
})
```

- Ajoutez le champ stripePaymentId et paidAt au modele Booking
- Ne jamais stocker les numeros de carte — Stripe gere tout
- Utiliser uniquement les cle TEST Stripe

5.2 — Ecran Paiement Mobile (10 pts)

Creez un ecran de paiement professionnel :

- Recapitulatif de la commande (service, date, adresse, montant)

ELITEFORCE.GLOBAL — TEST TECHNIQUE SENIOR

React Native + Node.js + PostgreSQL + AWS — Projet
MultiServices

Duree : 8 heures minimum
Total : 200 points | Seuil : 140/200
Livraison : 48h apres reception

- CardField de Stripe (formulaire carte integre)
- Bouton 'Payer X Dhs' desactive pendant le chargement
- Indicateur de securite (icone cadenas + 'Paiement securise par Stripe')
- Ecran de confirmation apres paiement reussi
- Ecran d'erreur avec message clair si paiement echoue
- Gestion des cartes de test :
 - 4242 4242 4242 4242 → Paiement reussi
 - 4000 0000 0000 9995 → Fonds insuffisants
 - 4000 0025 0000 3155 → Authentification 3D Secure requise

5.3 — Securite Backend (15 pts)

Implementez ces mesures de securite OBLIGATOIRES :

Rate Limiting :

```
npm install express-rate-limit
```

- Maximum 100 requetes / 15 minutes par IP (global)
- Maximum 5 tentatives de connexion / heure par IP

Validation des donnees :

```
npm install express-validator
```

- Valider TOUTES les entrees utilisateur
- Sanitiser les chaines de caracteres (XSS)
- Exemple : email doit etre un email valide, prix doit etre positif

Headers de securite :

```
npm install helmet
```

- Ajouter helmet() comme middleware global
- Configurer CORS strictement (autoriser seulement votre domaine)

Mots de passe :

```
npm install bcryptjs
```

- Hash avec bcrypt, saltRounds = 12 minimum
- Ne jamais retourner le hash du mot de passe dans les reponses

JWT :

- Expiration a 7 jours
- Refresh token mechanism (bonus)
- Verifier l'expiration dans le middleware authenticate

CRITERE D'EVALUATION	DESCRIPTION	POINTS
PaymentIntent backend	Creation correcte avec montant et metadata	5 pts

ELITEFORCE.GLOBAL — TEST TECHNIQUE SENIOR

React Native + Node.js + PostgreSQL + AWS — Projet
MultiServices

Duree : 8 heures minimum
Total : 200 points | Seuil : 140/200
Livraison : 48h apres reception

Confirmation paiement mobile	Flux Stripe complet et fonctionnel	5 pts
Webhook Stripe	Verification signature + mise a jour booking	5 pts
Ecran paiement UI	CardField + recapitulatif + etats loading/succes/erreur	10 pts
Rate limiting	Global + login implementes	5 pts
Validation inputs	Tous les endpoints valides avec express-validator	5 pts
Helmet + CORS	Headers securite + CORS configure	3 pts
Bcrypt saltRounds >= 12	Mot de passe correctement hashe	2 pts

ELITEFORCE.GLOBAL — TEST TECHNIQUE SENIOR

React Native + Node.js + PostgreSQL + AWS — Projet
MultiServices

Duree : 8 heures minimum
Total : 200 points | Seuil : 140/200
Livraison : 48h apres reception

MODULE 6

Déploiement & Documentation Technique

DUREE
30 min

POINTS
20 pts

NIVEAU
Expert

Contexte

Un developpeur senior sait deployer son travail et le documenter. Ce module evalue votre capacite a livrer quelque chose de deployable, pas juste du code en local.

6.1 — Variables d'environnement et Configuration (5 pts)

- Aucune cle secrete committee dans le repo (utiliser .env)
- Fichier .env.example complet et a jour
- Configuration Prisma pour differents environnements (dev/prod)
- Seeds Prisma : 10 services pre-remplis pour tester l'app

```
// backend/prisma/seed.js — Structure attendue
const services = [
  { name: 'Menage complet', category: 'Menage', basePrice: 150, duration: 120 },
  { name: 'Plomberie urgence', category: 'Plomberie', basePrice: 300, duration:
60 },
  // ... 8 autres services
]
```

6.2 — Docker (5 pts)

Creez un Dockerfile pour le backend :

```
// backend/Dockerfile — Structure attendue
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npx prisma generate
EXPOSE 3000
CMD ['node', 'src/index.js']
```

Et un docker-compose.yml a la racine :

```
version: '3.8'
services:
  backend:
    build: ./backend
    ports: ['3000:3000']
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/eliteforce
    depends_on: [db]
```

ELITEFORCE.GLOBAL — TEST TECHNIQUE SENIOR

React Native + Node.js + PostgreSQL + AWS — Projet
MultiServices

Duree : 8 heures minimum
Total : 200 points | Seuil : 140/200
Livraison : 48h apres reception

```
db:
  image: postgres:15-alpine
  environment:
    - POSTGRES_DB=eliteforce
    - POSTGRES_USER=user
    - POSTGRES_PASSWORD=pass
```

6.3 — Documentation API (Postman ou Swagger) (5 pts)

Documentez TOUS vos endpoints. Choisissez UNE des options :

OPTION A — Collection Postman :

- Exportez votre collection Postman en JSON
- Committez le fichier eliteforce-api.postman_collection.json
- Chaque endpoint avec exemple de requete et reponse

OPTION B — Swagger/OpenAPI :

```
npm install swagger-ui-express swagger-jsdoc
```

- Documentation accessible sur /api/docs
- Tous les endpoints documentes avec schemas

6.4 — README Final de Qualite (5 pts)

Le README doit permettre a quelqu'un de lancer le projet en 10 minutes. Incluez :

- Badges : Node version, License, Tests
- Architecture diagram (meme en ASCII)
- Instructions complete : clone → install → .env → migration → seed → launch
- Screenshots de l'application (minimum 5 ecrans)
- Liste des endpoints API avec exemples
- Guide de contribution (comment ouvrir une PR)

CRITERE D'EVALUATION	DESCRIPTION	POINTS
Seeds Prisma fonctionnels	10 services pre-remplis correctement	5 pts
Dockerfile valide	Image buildable et fonctionnelle	3 pts
docker-compose fonctionnel	Backend + PostgreSQL lancent avec docker-compose up	2 pts
Documentation API	Collection Postman OU Swagger complete	5 pts
README professionnel	Installation en 10 min sans aide	5 pts

ELITEFORCE.GLOBAL — TEST TECHNIQUE SENIOR

React Native + Node.js + PostgreSQL + AWS — Projet
MultiServices

Duree : 8 heures minimum
Total : 200 points | Seuil : 140/200
Livraison : 48h apres reception

BAREME DE SELECTION FINAL

SCORE	NIVEAU	DECISION	SUITE
180 — 200	Exceptionnel	RETENU IMMEDIATEMENT	Contrat envoye sous 24h
160 — 179	Excellent	ENTRETIEN PRIORITAIRE	Appel video sous 48h
140 — 159	Tres bon	ENTRETIEN PLANIFIE	Discussion sous 72h
120 — 139	Insuffisant	NON RETENU	Feedback envoye par email
Moins de 120	Faible	ELIMINE	Pas de suite donnee

INSTRUCTIONS DE LIVRAISON

CHECKLIST AVANT D'ENVOYER

- ✓ Repo GitHub public et accessible
- ✓ Envoyez lien GitHub a master.eliteforce.app@gmail.com
- ✓ Les deux dossiers mobile/ et backend/ sont presents
- ✓ Le backend se lance avec : `cd backend && npm install && npx prisma migrate dev && npm start`
- ✓ Le mobile se lance avec : `cd mobile && npm install && npx expo start`
- ✓ Le fichier `.env.example` est complet
- ✓ Le README permet de lancer le projet sans aide
- ✓ Aucune cle secrete dans le code (pas de `.env` commite)
- ✓ Minimum 10 commits avec des messages descriptifs

Envoyez le lien GitHub a master.eliteforce.app@gmail.com avec l'objet :

```
[TEST SENIOR] Prenom Nom — React Native Developer —  
[Ville/Pays]
```

Delai de livraison : 48 heures apres reception de ce document.

Bonne chance — Nous attendons votre meilleur travail.
master.eliteforce.app@gmail.com | eliteforce.global